

Software Requirements Specification

Use Case Based Capstone Template

Document field	Team entry
Project name	BetterWill
Team name and members	Team f Parker Giselle David Cameron
Sponsor and instructor	Commerce Bank mentors and Syed Jawad Hussain Shah
Version and status	0.1 Draft
Date	9/28/26
Requirements lead	David + Giselle

How to use this template

This specification contains functional and nonfunctional requirements. Use cases are the primary method for gathering and organizing functional requirements. Nonfunctional requirements define measurable qualities and constraints that apply to the use cases or the product as a whole. Complete both categories before deriving sprint stories, tests, and release decisions. Text inside brackets [] tells you what to write. Replace it with your own information before submitting the document.

- Use stable identifiers such as UC-AUTH-01, BR-001, NFR-SEC-001, US-AUTH-03, and TC-AUTH-004.
- Write use cases from the actor's goal and the system's observable response. Avoid screen layout and database design unless they are true constraints.
- Use FR identifiers for separate functional statements and NFR identifiers for measurable quality or constraint statements.
- Keep the requirements document as the behavioral source. Link sprint stories and test cases back to a use case flow.
- Record approved changes in the revision history and update traceability during every sprint.

Revision history

Version	Date	Author	Change and approval
0.1	10/2/26	Parker Giselle David Cameron	Initial actor list, use cases, and catalog

Contents

- 1 Introduction
- 2 Stakeholders Actors and Goals
- 3 Product Context and Constraints
- 4 Use Case Model
- 5 Functional Requirements and Detailed Use Cases
- 6 Nonfunctional Requirements
- 7 Data and External Interfaces
- 8 Traceability and Sprint Mapping
- 9 Review and Change Control
- Appendix A Reusable Detailed Use Case Form

1 Introduction

1.1 Purpose and audience

This document specifies the scope and requirements of the Betterwill project application. This document will be reviewed by the developers of the application at set intervals during development. This document will support design by specifying the requirements of the application and guide the developers by providing concrete development goals. Sponsors can use this document to ensure the goals are developed as promised.

1.2 Problem and product vision

Charitable organizations, with an emphasis on organizations like shelters, have specific needs that they need fulfilled. These needs, like clothing and food, are satisfied by charitable donations from individuals or organizations. Our application seeks to personalize these donations by allowing the charitable organizations to specify their needs and for the users to purchase items to satisfy these needs.

1.3 Scope

In scope	Out of scope
User Features • Account Registration • User Login • User Profiles • User Settings • Donation History • Dashboard Donation Features • Browse wish-list items • Sponsor specific items • Fund entire wish lists • Progress tracking • Donation history • Donation analytics Administrative Features • Create wish-list items • Update fulfillment status • Manage donations • Run reports • View financial ledger	Real banking connections, real money movement, and real card data. Live integration with online stores: item vendors and their catalogs are simulated with fabricated data, and no real orders are placed.

In scope	Out of scope
Users will provide dummy payment information, and the application will use that information to make dummy purchases for the benefit of a specified organization	Real banking connections, real money movement, and real card data unless the sponsor explicitly changes the brief

1.4 Definitions and conventions

Term	Definition
Actor	A person, role, device, or external system outside the product boundary that interacts with the product.
Use case	A goal-oriented interaction between an actor and the system.
Scenario	One path through a use case, including the main success path or an alternate or exception path.
Functional requirement	A capability, rule, validation, calculation, or system response that the product must provide.
Nonfunctional requirement	A measurable quality attribute or constraint on how the product operates or is built.
Must	Mandatory requirement.
Should	Desired requirement that may be deferred with an approved reason.

1.5 Assumptions dependencies and open questions

ID	Type	Statement	Owner	Target resolution
A-001	Assumption	The team will be able to integrate online stores like Amazon into the application.	David	10/2/26
D-001	Dependency	Reliance on the previously mentioned online stores for the full functionality of the application.	David	10/2/26
D-002	Dependency	Any delivery of goods will be accomplished by external package delivery services.	David	10/2/26
Q-001	Open question	Will the sponsor allow us to integrate real online stores into our prototype.	David	10/2/26

2 Stakeholders Actors and Goals

Identify people and external systems before naming use cases. An actor represents a role, not a specific teammate or screen.

2.1 Stakeholders

Stakeholder	Interest or decision	How requirements will be validated
Commerce Bank mentors	Sponsor alignment and technical feedback	Midsemester review, questions, and final walkthrough

Stakeholder	Interest or decision	How requirements will be validated
Instructor	Course expectations and evaluation	Scheduled reviews and submission feedback

2.2 Actors and user characteristics

Actor ID	Actor or role	Goals	Knowledge permissions and needs
ACT-01	Registered user	[Primary goals]	[Skill level, permissions, accessibility or usability needs]
ACT-02	Prospective user	[Goals or service provided]	[Constraints and trust assumptions]
ACT-03	Organizations		

2.3 Actor goal summary

Actor	Goal	Success outcome	Candidate use case
Registered user	Create a monthly budget	Budget is stored for that user and shown in totals	UC-BUD-01
[Actor]	[Goal stated with a verb]	[Observable outcome]	[UC-ID]

3 Product Context and Constraints

3.1 Product environment and system boundary

BetterWill is a browser-based web application that can be accessed on desktop. The front-end will consist of using Tailwind for the CSS framework and React for the library. The server/API uses Node along with PostgreSQL for database handling. There are 4 primary APIs: authentication is registration, login and password hashing; donation is making and tracking donations; wishlist is creating and managing wish-list items; reporting is the data behind analytics and transparency dashboards. A constraint is the payment processor used to make payments, banks, computer systems, hosting softwares and tools, internet connection, ecommerce delivery, beneficiaries and team.

3.2 Sponsor brief baseline

- Build a full-stack web application selected by the team; do not use a CMS such as WordPress.
- Include a meaningful financial component such as budget tracking, simulated payments, or investments.
- Support user registration and login; store passwords as secure hashes, never as plain text.
- Use a database, a front-end framework or library, and one CSS framework.
- Provide Login/Register, Home/Dashboard, User Profile/Settings, and at least two additional pages.
- Use only fake or dummy card and financial data when simulating transactions.
- Include unit testing; automated tests are strongly encouraged.
- Choose and complete at least two stretch goals from the sponsor brief.

3.3 Chosen stretch goals

Stretch goal	Reason for selection	Use cases or requirements affected	Evidence
Accessibility Audit and fix errors	More users and engagement and high feasibility through UX skills.	NFR-ACC-001, NFR-UI-001, NFR-UI-002	Will be written in a report
Self-hosted Authentication	Users can safely login easier. The team has little experience.	UC-AUTH-01, UC-AUTH-02, NFR-SEC-001, NFR-REL-001	Planned testing

3.4 Business rules

Rule ID	Rule	Source	Related use cases
BR-001	Development, testing, and demonstrations must use fabricated financial and card data.	Sponsor brief	UC-WSH-03
BR-002	Partner organizations or administrators shall create/change wish-list items.	Team decision	UC-WSH-01
BR-003	Donations must not exceed the remaining amount needed per item and capped.	Team decision	UC-WSH-03
BR-004	Personal donation history shall only be seen by the donor who made them, except administrator.	Team decision	N/A
BR-005	Donation history shall only be seen by partner organizations for their own donations.	Team decision	N/A
BR-006	Administration shall view all donations and history.	Team decision	N/A
BR-007	Only created accounts shall be allowed to make donations.	Team decision	UC-AUTH-01, UC-WSH-03
BR-008	No donations can be refunded or cancelled, unless discretion by the administrator.	Team decision	N/A
BR-009	Users shall buy donated items using item vendors.	Team decision	UC-WSH-02

4 Use Case Model

4.1 Use case diagram

Insert a readable diagram that shows actors outside the system boundary and goal-oriented use cases inside it. Use include or extend only when the relationship improves understanding.

4.2 Use case catalog

ID	Use case name	Primary actor	Priority	Status	Target	Owner
UC-AUTH-01	Register an account	Prospective user	Must	Draft	Sprint 0	Parker
UC-AUTH-02	Sign in	Registered user	Must	Draft	Sprint 0	Giselle
UC-WSH-01	Edit wishlist item	Organization	Must	Draft	Sprint 1	David

ID	Use case name	Primary actor	Priority	Status	Target	Owner
UC-WSH-02	Fulfill wishlist item	Registered user	Must	Draft	Sprint 1	David
UC-WSH-03	Contribute funding towards a goal	Registered User	Must	Draft	Sprint 1	Cameron

5 Functional Requirements and Detailed Use Cases

Use cases are the primary source of functional requirements. Copy the reusable form in Appendix A for every high-priority use case. Complete the main success scenario and realistic alternate and exception flows before the related story enters a sprint. Add a separate FR statement when behavior applies across use cases or needs more precision than a flow step provides.

5.1 Detailed Use Cases

UC-AUTH-01 Register an Account

Field	Team entry
Use case ID and name	UC-AUTH-01 Register an Account
Goal	Create a new account
Primary actor	ACT-02 Prospective user
Supporting actors	None
Trigger	Register account chosen from sign in screen
Preconditions	Must have access to program Sign in page usable Account database established
Success postconditions	New account registered and in database
Minimum guarantee	Accounts database not affected on fail Program still running after fail
Priority status target owner	Must; Draft; Sprint 0

Main success scenario

Step	Actor or system	Action or response
1	Actor	User clicks to register account
2	System	Register page loaded
3	Actor	User inputs new username and password
4	System	System verifies new credentials

Step	Actor or system	Action or response
5	System	New credentials added to user database
6	System	Successful register message displayed to user

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
4a	4	New credentials not accepted (invalid input)	New credentials not sent, register page reset, failure message displayed
5a	5	Database failure	Partial data cleansed, register page reset, failure message displayed

Rules data quality and evidence

Item	Linked IDs or details
Business rules	BR-001
Data inputs and outputs	User provided credentials, a confirmation message
Functional requirements	FR-AUTH-001
Nonfunctional requirements	NFR-SEC-001
Acceptance and test evidence	[TC IDs, Given/When/Then examples, automated-test links]
Backlog mapping	[Story or task IDs and the specific scenario slice each implements]

UC-AUTH-02 Sign In

Field	Team entry
Use case ID and name	UC-AUTH-02 Sign in
Goal	Sign into an existing account
Primary actor	ACT-01 Registered user
Supporting actors	None
Trigger	Sign in selected from main page
Preconditions	Must have access to program Sign in page usable Account database established
Success postconditions	User logged into their account

Field	Team entry
Minimum guarantee	Accounts database not affected on fail Program still running after fail User remains not logged in after fail
Priority status target owner	Must; Draft; Sprint 0

Main success scenario

Step	Actor or system	Action or response
1	Actor	User clicks on sign in
2	System	Sign in page loaded
3	Actor	User inputs existing username and password
4	System	System verifies credentials
5	System	Successful log in message displayed to user

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
4a	4	Credentials not accepted (credentials do not exist in the database)	User not logged in, log in page reset, failure message displayed

Rules data quality and evidence

Item	Linked IDs or details
Business rules	None
Data inputs and outputs	User provided credentials, a confirmation message
Functional requirements	FR-AUTH-001
Nonfunctional requirements	NFR-USE-001, NFR-REL-001
Acceptance and test evidence	[TC IDs, Given/When/Then examples, automated-test links]
Backlog mapping	[Story or task IDs and the specific scenario slice each implements]

UC-WSH-01 Edit Wishlist Item

Field	Team entry
Use case ID and name	UC-WSH-01 Edit wishlist item
Goal	Create, edit or delete wishlist items

Field	Team entry
Primary actor	ACT-03 Organization
Supporting actors	None
Trigger	Manage wishlist selected from the user dashboard
Preconditions	Must have access to program User dashboard usable Organization cleared to make a wishlist User is logged into their account
Success postconditions	Wishlist reflects change made
Minimum guarantee	Wishlist not affected on fail Program still running after fail
Priority status target owner	Must; Draft; Sprint 1

Main success scenario

Step	Actor or system	Action or response
1	Actor	User clicks on edit wishlist
2	System	Wishlist loaded
3	Actor	User clicks create listing
4	System	System brings up listing creation page
5	Actor	User creates the listing based on their needs
6	System	Listing is added to wishlist and is visible to registered users

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
3a	3	User clicks on edit listing	User presented with edit page, and desired changes are made
3b	3	User clicks on delete listing	The listing is removed from the user dashboard and is no longer visible to other registered users

Rules data quality and evidence

Item	Linked IDs or details
Business rules	None
Data inputs and outputs	User provided information to be entered into the listing, a confirmation message
Functional requirements	FR-AUTH-001

Item	Linked IDs or details
Nonfunctional requirements	NFR-UI-002
Acceptance and test evidence	[TC IDs, Given/When/Then examples, automated-test links]
Backlog mapping	[Story or task IDs and the specific scenario slice each implements]

UC-WSH-02 Fulfill Wishlist Item

Field	Team entry
Use case ID and name	UC-WSH-02 Fulfill wishlist item
Goal	Purchase an item from a wishlist
Primary actor	ACT-01 Registered user
Supporting actors	None
Trigger	Buy now selected from wishlist
Preconditions	Must have access to program User logged in Wishlist created
Success postconditions	Purchase registered and made
Minimum guarantee	Program still running after fail
Priority status target owner	Must; Draft; Sprint 1

Main success scenario

Step	Actor or system	Action or response
1	Actor	User buy now on wishlist
2	System	Brings up order page
3	Actor	User selects from saved payment information
4	System	Payment is processed via payment processor
5	System	Payment is successful and purchase is made

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
3a	3	No payment information saved	User enters payment information and is prompted to save the information

Rules data quality and evidence

Item	Linked IDs or details
Business rules	BR-008
Data inputs and outputs	User provided payment information, a confirmation message
Functional requirements	FR-AUTH-001
Nonfunctional requirements	NFR-PRIV-001
Acceptance and test evidence	[TC IDs, Given/When/Then examples, automated-test links]
Backlog mapping	[Story or task IDs and the specific scenario slice each implements]

UC-WSH-03 Contribute funding towards a goal

Field	Team entry
Use case ID and name	UC-WSH-03 Contribute funding towards a goal
Goal	Pay part of a large purchase
Primary actor	ACT-01 Registered user
Supporting actors	None
Trigger	User selects buy in part on wishlist
Preconditions	Must have access to program User logged in Wishlist created
Success postconditions	Funds successfully moved
Minimum guarantee	User not charged on fail Program still running after fail
Priority status target owner	Must; Draft; Sprint 1

Main success scenario

Step	Actor or system	Action or response
1	Actor	User clicks on buy in part in wishlist
2	System	System brings up payment page
3	Actor	User selects from saved payment information and selects the desired amount
4	System	Payment is processed via payment processor
5	System	Payment is successful and funds are moved

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
3a	3	No payment information saved	User enters payment information and is prompted to save the information

Rules data quality and evidence

Item	Linked IDs or details
Business rules	BR-008
Data inputs and outputs	User provided payment information, a confirmation message
Functional requirements	FR-AUTH-001
Nonfunctional requirements	NFR-PRIV-001
Acceptance and test evidence	[TC IDs, Given/When/Then examples, automated-test links]
Backlog mapping	[Story or task IDs and the specific scenario slice each implements]

5.2 Functional requirements catalog

Use this catalog for required behavior that applies across use cases or needs a precise standalone statement. Sponsor-brief capabilities such as required pages belong here, not in the nonfunctional section. Every functional requirement must link to at least one use case, business rule, or sponsor constraint and must be testable.

FR ID	Functional requirement	Source or related use cases	Priority	Verification
FR-AUTH-001	The system must allow a prospective user to create an account using the approved registration fields.	UC-AUTH-01	Must	TC-AUTH-001
FR-BUD-003	The system must reject a budget amount that is zero or negative and must not create a partial record.	UC-BUD-01 flow 3a	Must	TC-BUD-006
FR-___	[One required capability or behavior using must]	[UC, BR, or brief ID]	[Priority]	[Test ID or method]

6 Nonfunctional Requirements

State measurable quality attributes and constraints that apply across the product or to named use cases. Every NFR must include operating conditions, a target or threshold, and a verification method. Do not write vague statements such as the system must be fast or secure.

ID	Category	Requirement and measure	Verification	Priority
NFR-SEC-001	Security	Passwords must be stored only as salted hashes produced by the selected authentication library; plain-text passwords must not appear in storage or logs.	Integration inspection and log review	Must
NFR-PRIV-001	Privacy	The application and all demonstrations must use fabricated financial and card data and must not request real banking credentials.	Repository database and demo review	Must
NFR-USE-001	Usability	Users must be able to sign in and see their information.	Human demo	Must
NFR-PERF-001	Performance	Bring up a list of wishlist items from the database after a search, within 10 seconds.	Database response time, log review, demo	Should
NFR-ACC-001	Accessibility	User should be able to navigate the software using keyboard and mouse; search boxes and buttons should have good contrast visibility	Human demo	Must
NFR-REL-001	Reliability	Failed login should give a failure message and allow retry without stopping the program.	Database logs and demo review	Should
NFR-MNT-001	Maintainability	Test cases and unit test should be made for program sections to ensure no loss of usability during development	Unit testing, log review, human demo	Should

6.1 Additional interface quality requirements

NFR ID	Quality requirement and measure	Related use cases	Verification
NFR-UI-001	For the selected representative workflows, all interactive controls must be reachable and operable by keyboard.	All	Keyboard walkthrough
NFR-UI-002	Labels, validation messages, and navigation must be understandable to representative users completing the selected tasks.	UC-AUTH-02 UC-WSH-01	Usability test with stated success criterion

7 Data and External Interfaces

7.1 Data requirements

Data ID	Entity or data	Important fields and validation	Ownership privacy and retention	Related use cases
DATA-USER	User account	[Fields and validation; password stored only as a hash; transactions; user ID; role]	[Owner, sensitivity, retention, deletion]	UC-AUTH-01 UC-AUTH-02
DATA-ORG	Organization Account	[Fields and validation; password stored only as a hash; wishlist; org ID]	[Owner, sensitivity, retention, deletion]	UC-AUTH-01 UC-AUTH-02
DATA-TRANS	Transaction Data	[Trans ID; item; price; user ID]	[Sensitivity, retention]	UC-WSH-02 UC-WSH-03
DATA-WISH	Wishlist Item	[Wish ID; Desc; price; org ID; transactions]	[Owner, sensitivity, retention]	UC-WSH-01 UC-WSH-02

7.2 External interfaces

Interface ID	System or service	Purpose and contract	Authentication data and failure behavior	Owner
INT-001				
INT-002				

8 Traceability and Sprint Mapping

8.1 Requirements traceability matrix

Use case or requirement	Scenario or statement	Story task	Design component	Test case	Status
UC-AUTH-01	Main flow: donor registers an account	US-AUTH-01	Registration page, Authentication API, Users table	TC-AUTH-001	Planned
UC-AUTH-01	Flow 4a: invalid credentials	US-AUTH-01	Registration validation and error message	TC-AUTH-002	Planned
UC-AUTH-01	Flow 5a: database failure	US-AUTH-01	Authentication API error handling	TC-AUTH-003	Planned
UC-AUTH-02	Main flow: registered user signs in	US-AUTH-02	Sign-in page, Authentication API	TC-AUTH-004	Planned
UC-AUTH-02	Alternate flow: failed sign-in, retry (NFR-REL-001)	US-AUTH-02	Sign-in error message	TC-AUTH-005	Planned
UC-WSH-01	Main flow: organization edits a wish item (BR-002)	US-WSH-01	Wish-list form, Wishlist API	TC-WSH-001	Planned
UC-WSH-03	Main flow: user contributes funding to a goal	US-WSH-03	Donation form, Donation API	TC-WSH-002	Planned
UC-WSH-03	Alternate flow: amount exceeds remaining (BR-003)	US-WSH-03	Donation validation	TC-WSH-003	Planned
UC-WSH-02	Main flow: wish marked fulfilled	US-WSH-02	Wishlist API, item status	TC-WSH-004	Planned
UC-DASH-01	Main flow: donor views dashboard	US-DASH-01	Dashboard page, Reporting API	TC-DASH-001	Planned
UC-PROF-01	Main flow: user edits profile and settings	US-PROF-01	Profile and Settings pages	TC-PROF-001	Planned
UC-ADM-01	Main flow: administrator views all donations	US-ADM-01	Admin view, Donation API	TC-ADM-001	Planned

8.2 Sprint backlog mapping

Story or task	Use case slice	Acceptance criteria	Target sprint	Owner	Result
US-AUTH-01 Register account	UC-AUTH-01 main flow, flows 4a and 5a	Given a new user, when they submit valid credentials, then the account is saved with a hashed password; invalid input shows an error and saves nothing	Sprint 0	Parker	Planned
US-AUTH-02 Sign in	UC-AUTH-02 main flow and failed sign-in	Given a registered user, when they enter correct credentials, then they are signed in; wrong credentials show an error and allow retry	Sprint 0	Giselle	Planned
US-WSH-01 Edit wish item	UC-WSH-01 main flow	Given an organization, when it saves an item, then the item shows with its goal amount	Sprint 1	David	Planned
US-WSH-03 Contribute funding	UC-WSH-03 main flow and over-limit flow	Given a signed-in user, when they contribute fake card data to an item, then progress updates; an amount over the remaining total is capped or rejected	Sprint 1	Cameron	Planned
US-WSH-02 Fulfill wish	UC-WSH-02 main flow	Given a fully funded item, when its purchase is confirmed, then it is marked fulfilled	Sprint 1	David	Planned
US-DASH-01 Donor dashboard	UC-DASH-01 main success scenario	Given a donor with donations, when they open the dashboard, then their history and funding progress are shown	Sprint 2	Cameron	Planned
US-PROF-01 Profile and settings	UC-PROF-01 main success scenario	Given a logged-in user, when they edit their profile, then the changes are saved and shown	Sprint 3	Giselle	Planned
US-ADM-01 Manage donations	UC-ADM-01 main success scenario	Given an administrator, when they open the admin view, then all donations and history are visible	Sprint 3	Parker	Planned

8.3 Prioritization

Sprint 0 delivers registration and sign-in first, since every other feature requires a signed-in user. Sprint 1 follows with the wish-list use cases, which are the core of the product: editing wish items, contributing funding, and fulfilling wishes. Within Sprint 1, item creation must come before funding, and funding before fulfillment, so the owners will work in that order. Sponsor-mandatory pages (dashboard, profile and settings, and additional pages) are scheduled after the core features but before stretch goals. The accessibility audit and self-hosted authentication are scheduled last, once the mandatory work is complete.

9 Review and Change Control

9.1 Review checklist

- [] Every actor has at least one goal and every in-scope goal maps to a use case.
- [] Each detailed use case has a trigger, preconditions, main success scenario, alternate or exception flows, and postconditions.
- [] The use case describes behavior and outcomes without unnecessary screen or database design.
- [] Functional requirements describe testable behavior and use stable FR identifiers when standalone statements are needed.
- [] Nonfunctional security, privacy, usability, performance, accessibility, and reliability requirements include measurable targets and verification methods.

- [] Use cases, FRs, and NFRs link to sprint stories and test cases by stable identifier.
- [] All examples, fixtures, screenshots, and demonstrations use fabricated financial data.

9.2 Change requests and decisions

Change ID	Requested change and reason	Affected IDs	Decision and approver	Date sprint
CHG-001	[Request and business or technical reason]	[UC BR NFR DATA IDs]	[Approved deferred rejected and by whom]	[Date / sprint]

Appendix A Reusable Detailed Use Case Form

Copy this entire appendix section for each use case. Replace every bracketed prompt and delete rows that do not apply only after the team confirms they are unnecessary.

Field	Team entry
Use case ID and name	[UC-AREA-## Goal-oriented verb phrase]
Goal	[Actor value]
Primary actor	[Actor ID and role]
Supporting actors	[Roles or external systems]
Trigger	[Starting event]
Preconditions	[Required prior state]
Success postconditions	[State after success]
Minimum guarantee	[State preserved after failure]
Priority status target owner	[Entries]

Main success scenario

Step	Actor or system	Action or response
1	[Actor/System]	[Action or observable response]
2	[Actor/System]	[Action or observable response]
3	[Actor/System]	[Action or observable response]
4	[Actor/System]	[Action or observable response]
5	[Actor/System]	[Action or observable response]
6	[Actor/System]	[Action or observable response]

Alternate and exception flows

Flow ID	Related step	Condition	System behavior and recovery
__a	__	[Condition]	[Behavior and return point]
__b	__	[Condition]	[Behavior and return point]
__c	__	[Condition]	[Behavior and return point]

Rules data quality and evidence

Item	Linked IDs or details
Business rules	[BR IDs]

Item	Linked IDs or details
Data inputs and outputs	[Data IDs and validation]
Functional requirements	[FR IDs]
Nonfunctional requirements	[NFR IDs]
Acceptance and test evidence	[TC IDs and acceptance examples]
Open questions and assumptions	[Q and A IDs]
Backlog mapping	[Story or task IDs with scenario slices]

Use case review record

Review field	Entry
Owner and last updated	[Name and date]
Reviewers	[Requirements reviewer, developer, tester, sponsor or instructor when applicable]
Status and target	[Proposed / Draft / Approved; target sprint or release]
Decision or follow-up	[Approved, changes requested, deferred question, and linked issue IDs]

- [] The primary actor and goal are clear.
- [] Success, alternate, and failure behavior are testable.
- [] Business rules, functional requirements, nonfunctional requirements, sprint work, and tests are linked by ID.
- [] The use case is ready for the target sprint only after review issues are resolved.